

DRF2GO Backend

 Python 3.13

 Django 6.0

 DRF 3.18

 Channels 4.3

 Celery 5.6

 PostgreSQL 18

 RabbitMQ 4

 Redis 8

 nginx 1.30

 Daphne 4.2 ASGI

 Docker Compose v2

 deps Pipfile.lock

 lint & format Ruff

 tests pytest

 pre-commit enabled

 OpenAPI 3

 auth SimpleJWT

 tasks Makefile

A Django REST Framework service template: JWT auth, OpenAPI docs, Celery tasks, WebSockets, and a Docker setup that runs the same image in development and production.

```
make env      # create .env from .env.example
make up       # build and start the development stack
```

Then open <http://localhost:8000/api/schema/swagger-ui/>.

Contents

- [What's in the box](#)
- [Requirements](#)
- [Project layout](#)
- [Getting started](#)
- [Make targets](#)
- [Configuration](#)
- [Settings architecture](#)

- [Docker architecture](#)
- [Production deployment](#)
- [Code quality](#)
- [Testing](#)
- [Celery tasks](#)
- [WebSockets](#)
- [Logging](#)
- [Health probes](#)
- [Scaling notes](#)
- [Troubleshooting](#)
- [Upgrading from the previous layout](#)

What's in the box

Area	Choice
Framework	Django 6.0 · Django REST Framework 3.18
Auth	djanoorestframework-simplejwt with refresh-token rotation and blacklisting
API docs	drf-spectacular (OpenAPI 3, Swagger UI, ReDoc)
Database	PostgreSQL 18 via psycopg 3 (binary + pool extras)
Async tasks	Celery 5.6 · RabbitMQ 4 broker · Redis result backend · django-celery-beat
WebSockets	Django Channels 4.3 · channels-redis
Server	Daphne (ASGI) behind nginx 1.30
Dependencies	Pipfile / Pipfile.lock only
Lint & format	Ruff (replaces black, isort, flake8, autoflake)
Tests	pytest · pytest-django · coverage
Runtime	Python 3.13

Security defaults worth knowing about: [/admin/](#) is a honeypot that logs login attempts while the real admin lives at [ADMIN_URL](#); the settings module refuses to boot with a default [SECRET_KEY](#), a [guest](#) RabbitMQ password or a missing Redis password once [DEBUG](#) is off.

Requirements

- Docker 24+ with Compose v2 (everything below runs in containers)
 - GNU Make
 - Optional, for running Django directly on the host: Python 3.13 and [pipenv](#)
-

Project layout

```

drf2go/
├── .docker/                                # every Docker and Compose asset lives
here
│   ├── compose.yaml                        # development stack
│   ├── compose.prod.yaml                  # production stack
│   └── django/
│       ├── Dockerfile                     # multi-stage: base → venv →
development | production
│   ├── Dockerfile.dockerignore
│   └── entrypoint.sh                      # wait-for-db, migrate, collectstatic,
exec
│   ├── nginx/
│   │   ├── Dockerfile
│   │   ├── Dockerfile.dockerignore
│   │   └── nginx.conf                     # envsubst template
│   └── rabbitmq/
│       └── rabbitmq.conf
├── config/                                # Django project
│   └── app_configs.py                     # AppConfig overrides for third-party
apps
│   ├── asgi.py wsgi.py celery.py urls.py
│   └── settings/
│       ├── __init__.py                    # the canonical DJANGO_SETTINGS_MODULE
│       ├── base.py
│       ├── env.py                         # BASE_DIR + env loader
│       └── sub_settings/                  # celery, channels, cors, drf, logging,
│                                           # rabbitmq, redis, spectacular,
url_utils
├── core/                                  # shared app: tasks, consumers, views,
logging helpers
├── scripts/                              # operational smoke tests (Celery,
WebSocket)
├── tests/                                # pytest suite
├── Makefile                              # the entrypoint for everything
├── Pipfile / Pipfile.lock                # the only dependency manifest
├── pyproject.toml                         # ruff, pytest and coverage
configuration
└── .env.example                          # documented environment template

```

`Dockerfile.dockerignore` files sit next to each `Dockerfile`: BuildKit resolves `<dockerfile-path>.dockerignore` before falling back to the context root, which is what lets the ignore rules live under `.docker/` with everything else.

Getting started

```
git clone <repository-url> && cd drf2go
```

```
make env          # copy .env.example → .env, then edit the credentials
make up          # build images and start the stack in the foreground
```

Services and URLs:

URL	What
http://localhost:8000/	API root (Daphne)
http://localhost:8000/api/schema/swagger-ui/	Swagger UI
http://localhost:8000/api/schema/redoc/	ReDoc
http://localhost:8000/api/schema/	OpenAPI schema
http://localhost:8000/secret-admin/	Django admin (ADMIN_URL)
http://localhost:8000/admin/	Honeypot - logs unauthorised attempts
http://localhost:8000/healthz/	Liveness probe
http://localhost:8000/readyz/	Readiness probe (database + channel layer)
http://localhost:15672/	RabbitMQ management UI

Common follow-ups:

```
make superuser      # create an admin user
make logs SERVICE=web # tail one service
make manage ARGS="showmigrations" # any management command
make test           # run the suite
make down           # stop; `make destroy` also drops volumes
```

Running Django on the host

The stack is the supported path, but Django will run directly too:

```
pipenv sync --dev
pipenv shell
# point POSTGRES_HOST / RABBITMQ_HOST / REDIS_HOST at localhost in .env
first
python manage.py migrate
python manage.py runserver
```

Make targets

`make` on its own prints the full list. The important ones:

Target	Description
<code>make env</code>	Create <code>.env</code> from <code>.env.example</code> (never overwrites)
<code>make up</code> / <code>make up-d</code>	Build and start the dev stack, foreground / detached
<code>make down</code> / <code>make destroy</code>	Stop; <code>destroy</code> also removes volumes
<code>make logs SERVICE=web</code>	Tail logs
<code>make ps</code>	Container status
<code>make shell</code> / <code>make sh</code>	Django shell / POSIX shell in the web container
<code>make manage ARGS="..."</code>	Run any management command
<code>make migrate</code> / <code>make makemigrations</code> / <code>make superuser</code>	Common shortcuts
<code>make check</code>	<code>manage.py check --deploy</code>
<code>make lint</code> / <code>make lint-fix</code> / <code>make format</code> / <code>make format-check</code>	Ruff
<code>make lint-all</code>	Run every pre-commit hook over the tree
<code>make test ARGS="-k foo"</code> / <code>make test-cov</code>	pytest
<code>make smoke</code>	Celery + WebSocket smoke tests against the running stack
<code>make prod-build</code> / <code>make prod-up</code> / <code>make prod- down</code> / <code>make prod-destroy</code>	Production stack
<code>make prod-logs</code> / <code>make prod-ps</code> / <code>make prod- manage ARGS="..."</code>	Production operations
<code>make prod-config</code>	Render and validate the production compose file
<code>make clean</code> / <code>make prune</code>	Remove local artefacts / dangling Docker data

Two variables customise every target:

```
make up ENV_FILE=.env.staging # use a different environment file
make logs SERVICE=celery_worker # target one service
```

`ENV_FILE` is used both for Compose interpolation and as the containers' `env_file`, so one flag switches the whole environment.

The Makefile always passes `--project-directory .`, because the compose files live in `.docker/` but reference paths relative to the repository root. Use the targets rather than bare `docker compose` commands.

Configuration

Everything is environment-driven; `.env.example` documents every variable with its default. Highlights:

Core

Variable	Default	Notes
<code>SECRET_KEY</code>	insecure dev sentinel	Startup fails if left at the default with <code>DEBUG=False</code>
<code>DEBUG</code>	<code>True</code> (dev), forced <code>False</code> by the production stack	
<code>ALLOWED_HOSTS</code>	<code>localhost,127.0.0.1</code>	Comma-separated
<code>CSRF_TRUSTED_ORIGINS</code>	empty	Required for cross-origin admin/session POSTs
<code>ADMIN_URL</code>	<code>secret-admin/</code>	Real admin path; <code>/admin/</code> is the honeypot

Database

Variable	Default	Notes
<code>POSTGRES_DB</code> / <code>POSTGRES_USER</code> / <code>POSTGRES_PASSWORD</code>	-	Required by the stacks
<code>POSTGRES_HOST</code> / <code>POSTGRES_PORT</code>	<code>db</code> / <code>5432</code>	Set by the compose files
<code>DATABASE_URL</code>	unset	Optional override for a managed database
<code>DB_CONN_MAX_AGE</code>	<code>60</code>	Persistent connections (with health checks)

Connections are configured from **discrete parameters** rather than a URL, so a password containing `@`, `:`, `/` or `%` cannot corrupt a connection string.

Set `DATABASE_URL` only when you want to point at something outside the stack; the same applies to `RABBITMQ_URL`, `REDIS_URL` and `CELERY_BROKER_URL`.

Services

Variable	Default	Notes
RABBITMQ_USER / RABBITMQ_PASSWORD / RABBITMQ_VHOST	guest / guest / /	guest is rejected when <code>DEBUG=False</code>
REDIS_PASSWORD / REDIS_DB	dev-redis-password / 0	Required when <code>DEBUG=False</code>
CELERY_RESULT_BACKEND_DB	1	Redis database for task results
CELERY_WORKER_CONCURRENCY	4	Worker processes per container
CHANNEL_LAYER_IN_MEMORY	False	In-memory channel layer; single process only

HTTPS (applied only when `DEBUG=False`)

`SECURE_SSL_REDIRECT`, `SECURE_HSTS_SECONDS`, `SESSION_COOKIE_SECURE` and `CSRF_COOKIE_SECURE` all default to off so the bundled HTTP-only nginx works out of the box. Turn them on once TLS terminates in front of the stack - until then `SECURE_SSL_REDIRECT=True` produces a redirect loop. `SECURE_PROXY_SSL_HEADER` is set for you, so Django trusts nginx's `X-Forwarded-Proto`.

API and docs

`DRF_PAGE_SIZE`, `DRF_THROTTLE_ANON`, `DRF_THROTTLE_USER`, `JWT_ACCESS_TOKEN_LIFETIME_MINUTES`, `JWT_REFRESH_TOKEN_LIFETIME_DAYS`, `SPECTACULAR_TITLE`, `SPECTACULAR_DESCRIPTION`, `SPECTACULAR_VERSION`.

Settings architecture

`config.settings` is the single canonical value for `DJANGO_SETTINGS_MODULE` - `manage.py`, `wsgi.py`, `asgi.py`, Celery, pytest and both compose files all point at it. Import from `config.settings`, never `config.settings.base`.

```
config/settings/__init__.py    re-exports base
    └─ base.py                 core Django settings
        └─ sub_settings/
            ├── celery_conf.py   broker, result backend,
worker tuning                  ├── channels_conf.py    channel layer
                                ├── cors_headers_conf.py CORS
                                ├── drf_conf.py          DRF + SimpleJWT
                                ├── logging_conf.py       handlers, formatters,
loggers                        ├── rabbitmq_conf.py    broker connection
                                ├── redis_conf.py        Redis connection
                                └─ spectacular_conf.py   OpenAPI
```

Adding a feature area means adding one module under `sub_settings/` and one import line in its `__init__.py`. Both `redis_conf` and `rabbitmq_conf` accept either a URL or discrete parameters and normalise to both, using `url_utils.parse_url/build_url` (urllib-based, percent-encoding aware).

Register project apps in `LOCAL_APPS` in `base.py`.

Docker architecture

`.docker/django/Dockerfile` is one multi-stage build:

```
base          python:3.13-slim + curl + tini, entrypoint wired in
lockfile      renders Pipfile.lock → hash-pinned requirements (pipenv)
venv-prod     /opt/venv with runtime dependencies
venv-dev      /opt/venv with runtime + dev dependencies
development ← venv-dev, source bind-mounted, runs as root
production   ← venv-prod, source baked in, runs as uid 1001
```

Notes:

- Dependencies come **only** from `Pipfile.lock`. `pipenv verify` fails the build if the Pipfile and lock have drifted apart, and pip installs against the lock's hashes.
- The virtualenv lives at `/opt/venv`, outside `/app`, so the development bind mount cannot shadow it.
- `tini` is PID 1, so `docker compose stop` shuts Daphne and Celery down cleanly.
- uWSGI is gone. Daphne serves both HTTP and WebSockets; a WSGI server cannot, and the old `uwsgi.ini` was dead configuration the compose files overrode.

The entrypoint is shared by web, worker and beat, and is driven by environment variables:

Variable	Default	Effect
<code>WAIT_FOR_DB</code>	<code>true</code>	Block until the database accepts connections
<code>AUTO_MIGRATE</code>	<code>false</code>	Run <code>migrate</code> before starting
<code>AUTO_COLLECTSTATIC</code>	<code>false</code>	Run <code>collectstatic</code> before starting

Migrations and collectstatic default to **off** and are enabled on the `web` service only, so scaling to several replicas never races two `migrate` runs.

`.docker/rabbitmq/rabbitmq.conf` re-enables two features RabbitMQ 4 stopped permitting by default but that the Celery client still needs - `transient_nonexcl_queues` (Celery's control plane, including the worker healthcheck) and `global_qos` (how the worker applies its prefetch limit) - and raises the connection log category to `error` so the healthcheck's connection churn does not bury real messages.

Production deployment

```
cp .env.example .env      # then set real values - see below
make prod-up              # build and start, detached
make prod-ps
make prod-logs
```

Minimum production `.env` changes:

```
SECRET_KEY=<generate one>
ALLOWED_HOSTS=example.com,www.example.com
CSRF_TRUSTED_ORIGINS=https://example.com,https://www.example.com
POSTGRES_DB=drf2go
POSTGRES_USER=drf2go
POSTGRES_PASSWORD=<strong>
RABBITMQ_USER=drf2go
RABBITMQ_PASSWORD=<strong, not "guest">
REDIS_PASSWORD=<strong>
CORS_ORIGIN_ALLOW_ALL=False
CORS_ALLOWED_ORIGINS=https://example.com
SERVER_NAME=example.com
HTTP_PORT=80
```

Generate a secret key:

```
python -c "from django.core.management.utils import
get_random_secret_key; print(get_random_secret_key())"
```

The production stack differs from development by:

- building the `production` image target - no dev dependencies, unprivileged user
- no source bind-mount; the image is immutable
- nginx terminating HTTP and serving `/static/` and `/media/` from shared volumes
- `DEBUG=False`, which activates the credential validation described above
- backing services not published on the host
- `!?` guards on every credential, so a missing value stops the deploy

Compose refuses to start rather than accepting a missing credential:

```
error while interpolating services.web.environment.[]: required variable
POSTGRES_PASSWORD is missing a value: POSTGRES_PASSWORD is required
```

TLS

The bundled nginx listens on plain HTTP so the stack works before certificates exist. To go HTTPS: terminate TLS in front of nginx (or add a `listen 443 ssl` block to `.docker/nginx/nginx.conf`), then set `SECURE_SSL_REDIRECT=True`, `SESSION_COOKIE_SECURE=True`, `CSRF_COOKIE_SECURE=True` and a non-zero `SECURE_HSTS_SECONDS`. Verify with `make prod-manage ARGS="check --deploy"` - those four warnings are the only ones the default configuration produces.

Routine operations

```
make prod-manage ARGS="migrate"
make prod-superuser
make prod-logs SERVICE=celery_worker

# update
git pull && make prod-up

# backup / restore
docker compose --project-directory . -f .docker/compose.prod.yaml \
  exec db pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" > backup.sql
docker compose --project-directory . -f .docker/compose.prod.yaml \
  exec -T db psql -U "$POSTGRES_USER" "$POSTGRES_DB" < backup.sql
```

Code quality

Ruff is the single tool for linting **and** formatting. It replaces black (format), isort (**I**), flake8 (**E/F/W/B/C4**) and autoflake (**F401**, **F841**), and adds pyupgrade, flake8-bandit, flake8-django and flake8-simplify. All configuration is in `pyproject.toml` under `[tool.ruff]`.

```
make lint          # check
make lint-fix      # check and auto-fix
make format        # format
make format-check  # verify formatting in CI
```

Install the git hooks once:

```
make install      # pipenv sync --dev
make hooks        # pre-commit install
make lint-all    # run every hook over the whole tree
```

`.pre-commit-config.yaml` pins `ruff-check --fix` and `ruff-format` from `astral-sh/ruff-pre-commit` at the same version as the Pipfile, plus a few hygiene hooks (trailing whitespace, YAML/TOML validity, private-key detection).

Testing

```
make test          # whole suite
make test ARGS="-k websocket" # filter
make test ARGS="-m unit"    # by marker
make test-cov        # with coverage (term + XML)
```

pytest is configured in `pyproject.toml`: `DJANGO_SETTINGS_MODULE=config.settings`, `testpaths=["tests"]`, `--reuse-db`, `--strict-markers`, and the markers `slow`, `integration`, `unit` and `api`. Coverage settings live under `[tool.coverage.*]` and measure `config` and `core`.

Fixtures in `conftest.py`: `api_client`, `authenticated_api_client`, `user`, `superuser`.

```
import pytest

@pytest.mark.django_db
def test_list_requires_auth(api_client):
    assert api_client.get("/api/things/").status_code == 401
```

Celery tasks

Worker and beat run as separate services in both stacks. RabbitMQ is the broker; task results go to Redis (database `CELERY_RESULT_BACKEND_DB`, default `1`).

```
from core.tasks import simple_async_task

result = simple_async_task.delay("payload")
result.id # track it
result.get(timeout=10)
```

Define tasks in any installed app's `tasks.py` - autodiscovery picks them up. Periodic schedules use `django-celery-beat`'s `DatabaseScheduler`, so they are editable from the admin at runtime; entries declared in `config/celery.py` `beat_schedule` are synced into the database when beat starts.

Reliability defaults set in `celery_conf.py`: `acks_late=True`, `prefetch_multiplier=1`, `reject_on_worker_lost=True` and `max_tasks_per_child=1000`. Together these mean a task is only acknowledged once it completes, so killing a worker mid-task redelivers the message instead of dropping it - which is what makes adding worker replicas safe.

Smoke-test the whole path against a running stack:

```
make smoke
```

WebSockets

Consumers live in `core/consumers.py` and are routed in `core/routing.py`. The sample echo consumer is at `ws://localhost:8000/ws/simple/`.

```
# core/consumers.py
from channels.generic.websocket import AsyncWebsocketConsumer

class MyConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()

    async def receive(self, text_data):
        await self.send(text_data=text_data)
```

```
# core/routing.py
websocket_urlpatterns = [
    path("ws/simple/", SimpleConsumer.as_asgi()),
    path("ws/mine/", MyConsumer.as_asgi()),
]
```

Test it:

```
docker compose --project-directory . -f .docker/compose.yaml \
    exec web python scripts/test_websocket.py --url
ws://127.0.0.1:8000/ws/simple/
```

nginx proxies `/ws/` with `Upgrade/Connection` headers and a 24-hour read timeout, so WebSockets work identically through the production stack.

Browser clients: wrap the router in `channels.security.websocket.AllowedHostsOriginValidator` (see `config/asgi.py`) to reject cross-site WebSocket handshakes. It is off by default because it also rejects non-browser clients, which send no `Origin` header.

Logging

Console logging is on by default and goes to stdout, which is what container platforms want. File logging and JSON output are opt-in:

```
LOG_LEVEL=INFO          # DEBUG | INFO | WARNING | ERROR | CRITICAL
LOG_ALL_LEVELS=False    # force DEBUG everywhere
LOG_TO_CONSOLE=True
LOG_TO_FILE=False       # rotating files under LOG_DIR
LOG_FORMAT=verbose      # verbose | json
LOG_DIR=logs
LOG_FILE_MAX_BYTES=10485760
LOG_FILE_BACKUP_COUNT=5
LOG_DB_QUERIES=False    # log every SQL statement
```

With `LOG_TO_FILE=True` you get `application.log`, `error.log` and `database.log` in `LOG_DIR`, each rotated. `LOG_FORMAT=json` emits structured records via `python-json-logger`, which is what you want when shipping logs to an aggregator.

In application code:

```
from core.logging import get_logger

logger = get_logger(__name__)
logger.info("Order created", extra={"order_id": order.id})
```

`core/logging.py` also provides `log_request_response`, `log_execution_time`, `log_exception` and `enrich_log_context`.

Health probes

Endpoint	Meaning
<code>/healthz/</code>	Liveness. Touches nothing; answers as long as the ASGI app is serving. Used by the container healthchecks.

Endpoint	Meaning
/readyz/	Readiness. Verifies the database and the channel layer, returns 503 when either is down. Point your load balancer here.

```
{"status": "ok", "checks": {"database": "ok", "channel_layer": "ok"}}
```

Every service in both stacks has a healthcheck, and `depends_on` uses `condition: service_healthy` - so `make up` does not report success until the stack is genuinely serving traffic.

Scaling notes

Things in this setup that exist specifically so it scales horizontally:

- **One migrator.** Only `web` sets `AUTO_MIGRATE`; workers and beat wait for the database but never migrate.
- **Persistent database connections.** `CONN_MAX_AGE=60` with `CONN_HEALTH_CHECKS=True`.
- **Paginated by default.** `LimitOffsetPagination` with `DRF_PAGE_SIZE`, plus configured anon/user throttle rates.
- **Late task acknowledgement.** Killing a worker redelivers its task.
- **Stateless beat.** Schedules live in PostgreSQL, not in a local file.
- **Redis channel layer.** Works across processes; the in-memory layer does not.

Adding replicas:

```
docker compose --project-directory . -f .docker/compose.prod.yaml \
  up -d --scale celery_worker=4
```

Scaling `web` requires removing its fixed host port binding (or putting nginx in front of a service-discovery-aware upstream).

Troubleshooting

port is already allocated - another stack is using 5432, 5672, 15672 or 8000. Override the published ports:

```
make up WEB_PORT=8001
POSTGRES_PORT_PUBLISHED=5433 RABBITMQ_PORT_PUBLISHED=5673 \
  RABBITMQ_MGMT_PORT_PUBLISHED=15673 make up
```

ImproperlyConfigured: SECRET_KEY must be set... - `DEBUG=False` with the default key. Generate one (see [Production deployment](#)).

RABBITMQ_PASSWORD must be changed from the default 'guest' / REDIS_PASSWORD is required - the production credential checks. Set real values in `.env`.

required variable POSTGRES_PASSWORD is missing a value - Compose's `?:guard`. Your `ENV_FILE` is missing that variable.

Database connection failures - `make logs SERVICE=db`, then confirm the `POSTGRES_*` values match between the `db` service and the app. `make destroy` wipes the volume for a clean start.

RabbitMQ deprecation messages on startup - these three are expected and harmless:

- `transient_nonexcl_queues ... permitted per the configuration`
- `global_qos ... permitted per the configuration`
- `management_metrics_collection is deprecated` (still permitted by default; it backs the management UI, and explicitly permitting it only swaps one warning for another)

The first two are logged because `.docker/rabbitmq/rabbitmq.conf` deliberately re-enables them. If instead you see them at `[error]` level, or Celery reports `INTERNAL_ERROR - Feature 'transient_nonexcl_queues' is deprecated`, the container did not pick the file up - confirm the mount with `docker compose ... exec rabbitmq cat /etc/rabbitmq/conf.d/10-drf2go.conf`, and recreate rather than restart the container after editing it.

Static files 404 in production - check the `collectstatic` step in `make prod-logs SERVICE=web`. `/app/static` and `/app/media` are shared volumes that must be owned by uid 1001; both images create them that way, so a stale volume from an older build is the usual cause - `make prod-destroy` recreates them.

Migration drift warnings for a third-party app - `DEFAULT_AUTO_FIELD` is being applied to vendored migrations. Add an AppConfig override in `config/app_configs.py`, as done for `admin_honeypot`.

Upgrading from the previous layout

If you are coming from an earlier checkout of this project:

Before	Now
<code>requirements.txt</code> , <code>requirements-dev.txt</code>	Deleted - <code>Pipfile</code> / <code>Pipfile.lock</code> only
<code>black</code> , <code>isort</code> , <code>flake8</code> , <code>autoflake</code>	<code>Ruff</code>

Before	Now
Dockerfile, Dockerfile.dev	.docker/django/Dockerfile (targets development / production)
docker-compose.yml, docker-compose.prod.yml	.docker/compose.yml, .docker/compose.prod.yml
docker-entrypoint.sh	.docker/django/entrypoint.sh
nginx/	.docker/nginx/
.dockerignore	.docker/*/Dockerfile.dockerignore
uwsgi.ini, uWSGI	Removed - Daphne serves HTTP and WebSockets
DJANGO_SETTINGS_MODULE=config.settings.base	config.settings
psycopg2-binary	psycopg [binary, pool] 3
Celery results in rpc://	Redis (CELERY_RESULT_BACKEND_DB)
docker compose -f docker-compose.yml ...	make up (or make prod-up)

Breaking changes to plan for:

- **PostgreSQL 15 → 18.** The data directory format changed and the volume now mounts at `/var/lib/postgresql` instead of `/var/lib/postgresql/data`. Dump with the old image and restore into the new one; there is no in-place upgrade.
- **RabbitMQ 3 → 4** and **Redis 7 → 8**.
- **Django 5.2 → 6.0.** Pinned to the 6.0 series because `django-celery-beat` 2.9.0 requires `Django<6.1`.
- `CORS_ORIGIN_WHITELIST` was dropped; use `CORS_ALLOWED_ORIGINS`.
- The real admin path is now configurable via `ADMIN_URL` rather than hard-coded.

Contributing

1. Branch off `main`.
2. `make install && make hooks`.
3. Keep `make lint` and `make test` green.
4. Update `.env.example` and this README when you add configuration.